Multi-Agent UAV Capture and Evasion with Adversarial Reinforcement Learning

A Vision-Only Sim-to-Real Pipeline for Low-Cost Quadrotors

Owen Murphy and Rishikesh Narayana

Computer Systems Research Laboratory -(SysLab)

Dr. Gabor and Dr. Yilmaz, Period 7

May 2026

Table of Contents

Abstract

Adversarial pursuit-evasion is a hard control problem and a useful testbed for multi-agent reinforcement learning, with applications ranging from autonomous defense to search and rescue. We trained two competing policies, a tracker and an evader, to play a 3D pursuit-evasion game using PPO self-play in a custom PyBullet environment. Each agent observes the relative position and velocity of its opponent and commands its own 3D velocity. To produce realistic strategies, we used asymmetric observations between the two agents, a curriculum that gradually ramps up evader speed and introduces a cylindrical obstacle midway through training, and reward shaping that encourages efficient interception over reactive following. After training, the tracker consistently closes distance and intercepts the evader across randomized starting conditions, while the evader develops feinting and obstacle-using behaviors that make pursuit nontrivial. We then deployed the trained tracker on a DJI Tello quadrotor, replacing the simulated state with real-time AprilTag detection from the onboard camera and adding velocity prediction between vision frames, action smoothing, and a yaw controller to keep the target in view. The drone tracks a moving target in indoor flight, demonstrating that a policy trained entirely in simulation can transfer to physical hardware through a vision-based perception pipeline.

Keywords: reinforcement learning, proximal policy optimization, self-play, pursuit-evasion, sim-to-real transfer, AprilTag, quadrotor

I. Introduction

Autonomous tracking of an actively evasive aerial target is a control problem in which two agents with opposing objectives adapt to whatever strategy its opponent adopts, so any tracking law written around fixed assumptions about target motion loses validity the moment the evader starts training and vice versa. This makes adversarial pursuit a useful benchmark for multi-agent reinforcement learning, since the desired behavior is not a single optimal policy but a stable equilibrium between two policies that improve in response to one another.

Most published approaches to UAV pursuit-evasion remain in simulation. MEADDQN, OpenMARL, and PSO-M3DDPG report high capture rates in simulated arenas, but each presumes either privileged global state or sensor packages that are unavailable on consumer-grade quadrotors of the kind a small project can actually purchase. The closest example of a learned control policy that does survive transfer to physical hardware is the Swift system of Kaufmann et al. [11], which trained a control policy in simulation and flew it on a custom racing quadrotor against three human world champions in head-to-head races. Swift establishes that aggressive learned control is achievable on real aircraft, but it concerns time-trial racing on a static track, runs on substantially more capable hardware than what we use, and was assembled by a robotics laboratory at the University of Zurich.

Here we describe a learned pursuit-evasion system trained end-to-end in simulation and deployed on a DJI Tello EDU, an 87-gram commercial quadrotor with a unit price of approximately $130. The system consists of two stages joined by a single trained network. In the first stage, a tracker and an evader are trained against each other using Proximal Policy Optimization in a custom PyBullet environment, with asymmetric vision-limited observations, a curriculum that increases evader speed and introduces an obstacle partway through training, and reward shaping that pushes both agents off the degenerate strategies that otherwise emerge. In the second stage, the trained tracker is exported unchanged and runs on the Tello at 20 Hz, with relative-pose observations supplied by AprilTag detection from the onboard camera and a small set of additional components, an inter-frame velocity predictor, a low-pass action filter, and a yaw controller, that compensate for the latency and field-of-view differences between the clean simulator and a real latency-bound aircraft.

PPO self-play, fiducial-marker visual servoing, and curriculum-based training all appear in prior literature, so the contribution here lies less in any individual technique than in the specific composition of these techniques with the observation design and the deployment-side adaptations described in the sections that follow, which together are sufficient to produce a transferable pursuit policy on hardware that costs around $130. The trained tracker reaches a

final average tag rate of 95% in simulation after three million timesteps of self-play, and when loaded onto the Tello it tracks an AprilTag-mounted target drone in indoor flight, recovers from short occlusions, and reacquires the target after sudden direction changes.

II. Background

Reinforcement Learning

In a reinforcement learning setup, an agent interacts with an environment over discrete time steps, observing a state, picking an action from some defined action space, and getting back a scalar reward. The whole point is to learn a policy that maximizes the expected cumulative discounted reward over an episode, rather than imitating a labeled correct answer at each step the way a supervised model would. The learning signal is, in practice, sparse and delayed: a tracker drone in our setting might take dozens of timesteps of forward motion before any of those actions get credited with the eventual capture reward, which is part of what makes the problem hard [1]. Throughout this paper we use the standard vocabulary: the agent is the learner, the environment is everything the agent acts on, the action space is the set of available moves, the state is whatever the agent observes about the world at a given step, and the reward is the scalar feedback that drives policy updates.

Adversarial reinforcement learning extends this setup to two agents whose reward functions are opposed. Because each agent is part of the environment from which the other one learns, simultaneous optimization is unstable. The standard alternative is self-play, in which one agent is optimized while a frozen copy of the other generates opponent behavior, and the agents alternate the role of learner across rounds. Baker et al. [8] demonstrated that self-play of this form produces offensive and defensive strategies that emerge without being explicitly encoded into the reward, which is the property we depend on here: pursuit and evasion strategies are not behaviors we attempt to specify.

Proximal Policy Optimization

Vanilla policy-gradient methods are unstable in the presence of large updates: a single optimization step that moves the policy too far from its previous parameters can erase substantial accumulated training. Proximal Policy Optimization addresses this by clipping the probability ratio between the new and old policies, so that no single gradient step can move the policy outside a trust region [2]:

$$L(\theta) = E[\, \min(\, r(\theta)\hat{A},\ \mathrm{clip}(r(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}\,)\,]$$

In this expression, $r(\theta)$ is the probability ratio between the new and previous policies at a given state-action pair, $\hat{A}$ is the advantage estimate, and ε is the clip parameter. We use the Stable-Baselines3 implementation of PPO [3]. The policy and value networks are two-layer multilayer perceptrons of width 128, with a learning rate of 5e-4, discount factor $\gamma = 0.98$, GAE $\lambda = 0.95$, batch size 128, n_steps 512, n_epochs 10, entropy coefficient 0.05, and a continuous 3D velocity action space. We tested three variants of the PPO clip mechanism before committing to one. Vanilla PPO with a fixed clip range of 0.2 was unstable across our runs and frequently plateaued early. A Kullback-Leibler-divergence penalty in place of the clip term plateaued earlier still in the pursuit-evasion environment, although it gave the smoothest learning curve on the CartPole benchmark we used as a sanity check. Adaptive Clip PPO, in which the clip parameter is scaled by the standard deviation of the advantage estimate per the formulation of Chen et al., gave the best balance of stability and convergence speed in our setting, and is the variant used for the final policy.

Related Work

Prior work on UAV pursuit-evasion using reinforcement learning is concentrated almost entirely on the simulation side, with each published approach carrying an assumption that ends up preventing transfer to inexpensive hardware. MEADDQN [10] formulates the problem as a multi-agent extension of double deep Q-learning and obtains capture rates above 80% in simulation, but the observation space includes global position and orientation of every agent and the training time required is substantial. Modular multi-agent frameworks in the OpenMARL family [9] report higher capture rates still, though the training compute footprint and the per-scenario retraining requirement place them outside the budget of a small project. PSO-M3DDPG variants apply particle-swarm optimization to the policy parameters of a deep deterministic policy gradient agent; these share the global-state assumption with the deep Q-learning approaches and have not, to our knowledge, been demonstrated on physical hardware. What unites all three, and what makes them unsuitable as off-the-shelf solutions for the problem we set out to solve, is that none of them address the transfer to a real aircraft equipped with only a single forward-facing camera, which is the operating point that actually matters for a small platform like the Tello.

On the transfer side, the most informative comparison point is Swift [11]. Kaufmann et al. trained a model-free on-policy deep RL controller in simulation, augmented the simulator with empirical residual models of perception and dynamics estimated from real-world flight data, and showed that the resulting policy could race head-to-head against world-champion human pilots on a 75 m track at speeds exceeding 100 km/h. Two features of Swift carried directly into our system: the choice of network architecture and learning rate, and the observation that the

dominant cost of transfer lies not in the learning algorithm but in the interface between the policy and the noisy onboard perception. The dissimilarities are also relevant. Swift races a static gate sequence rather than a learning opponent, runs at a budget and on hardware that are not comparable to ours, and corrects for sim-to-real gap using residual models trained on motion-capture data we do not have.

Tools and Platform

Initial development of the simulator used Microsoft AirSim, an Unreal Engine UAV simulator with a Python and MAVLink interface and built-in domain randomization [4]. AirSim renders photorealistic imagery, but the throughput of the combined Unreal renderer and physics solver was insufficient to train at the scale we required. We migrated the training environment to PyBullet, an open-source physics engine [5], which trades visual fidelity for an order-of-magnitude speedup in rollout collection. The physical platform is the DJI Tello EDU, a 87-gram commercial quadrotor that exposes velocity-command control and a 720p H.264 video stream through a documented UDP interface; we drive it from Python through the djitellopy library [6]. Onboard perception is handled by AprilTag fiducial markers [7] detected with the pupil-apriltags library. We identified YOLOv5 [12] as the eventual replacement for AprilTag once marker-free drone detection becomes the bottleneck, but have not yet deployed it in the flight loop.

III. Applications

Counter-UAV operations are the most direct application we have in mind, since a tracker capable of maintaining visual lock on a maneuvering target under realistic perception conditions is precisely the closed-loop component of any system meant to intercept or shadow a rogue drone near sensitive airspace such as airports, stadiums, or military installations. Beyond that immediate use case, the same tracking behavior generalizes naturally to follow-mode work like aerial search and rescue, wildlife and asset monitoring, and inspection tasks where a small aircraft has to keep a moving subject in frame without a human pilot. The evader policy turns out to have an independent application, too, as an automated red-teaming target for evaluating other tracking systems; a learned hard target is far more repeatable across trials than asking a human pilot to fly evasively, and it sidesteps the safety problem of putting a person in front of an autonomous system that is still being debugged. More broadly, the recipe used here, training closed-loop control through self-play in simulation and then deploying it on inexpensive hardware via vision-based perception, generalizes to follow-me cameras, automated cinematography drones, and a wider class of robotics tasks in which the subject of control does not cooperate with the controller.

IV. Simulation Methodology

System Architecture

Figure 1 lays out the full pipeline. PPO self-play trains a tracker and an evader against each other inside a custom PyBullet environment with relative-coordinate observations and a continuous 3D velocity action space, and the trained tracker policy is then exported and run on the Tello at 20 Hz with the simulated opponent state replaced by an AprilTag-based perception pipeline. The network itself is not modified between the two stages; the perception pipeline and the deployment-side components described in Section V handle whatever discrepancies remain between simulation and hardware.

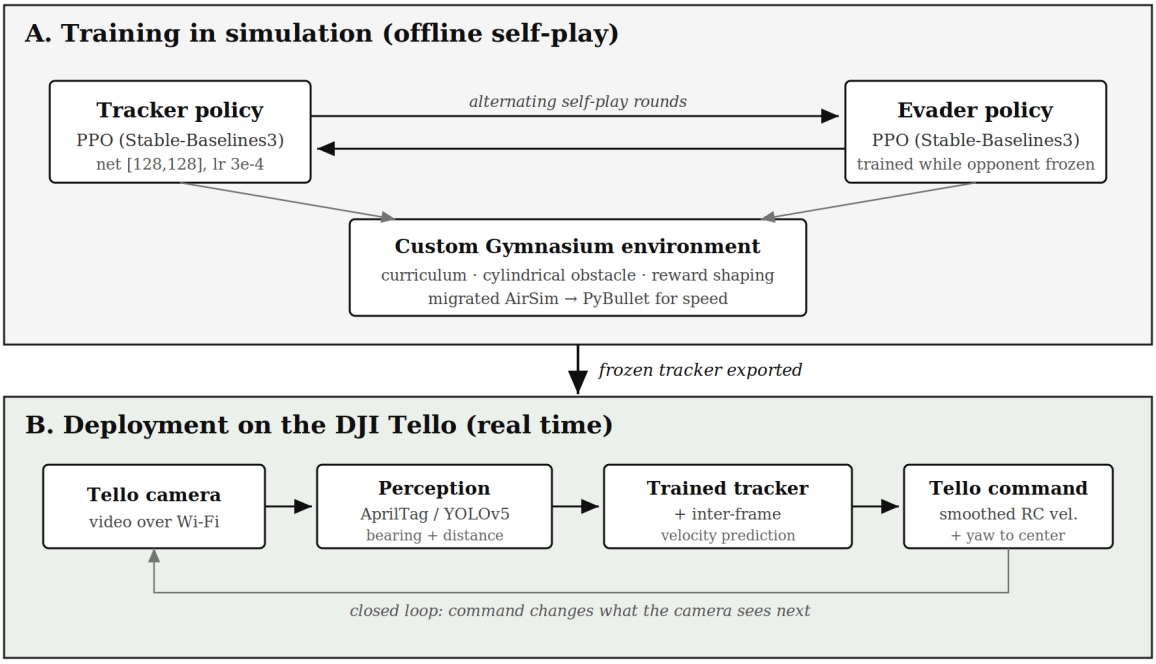

Figure 1. End-to-end architecture. PPO self-play trains the tracker and evader in PyBullet. The frozen tracker policy is then loaded into a Python control script that drives the Tello at 20 Hz, with AprilTag detection supplying the relative-pose component of the observation vector.

Observation and Action Spaces

Each agent observes a nine-dimensional vector composed of three components: the relative position of the opponent in the body frame of the observer, the observer's own linear velocity, and the opponent's estimated linear velocity. The action is a continuous three-dimensional velocity command bounded in [-1, +1] on each axis. Figure 2 illustrates these quantities for the tracker. The single most important design decision in the project, and the one that made transfer to hardware possible at all, was to express the observation in relative rather than absolute world coordinates. The Tello has no global positioning, so a policy trained on

absolute-coordinate observations would require a pose-estimation system we do not have. AprilTag detection produces a relative pose directly in the body frame of the camera, so the same observation vector that trained in simulation can be fed real detections without coordinate transformations or additional learning.

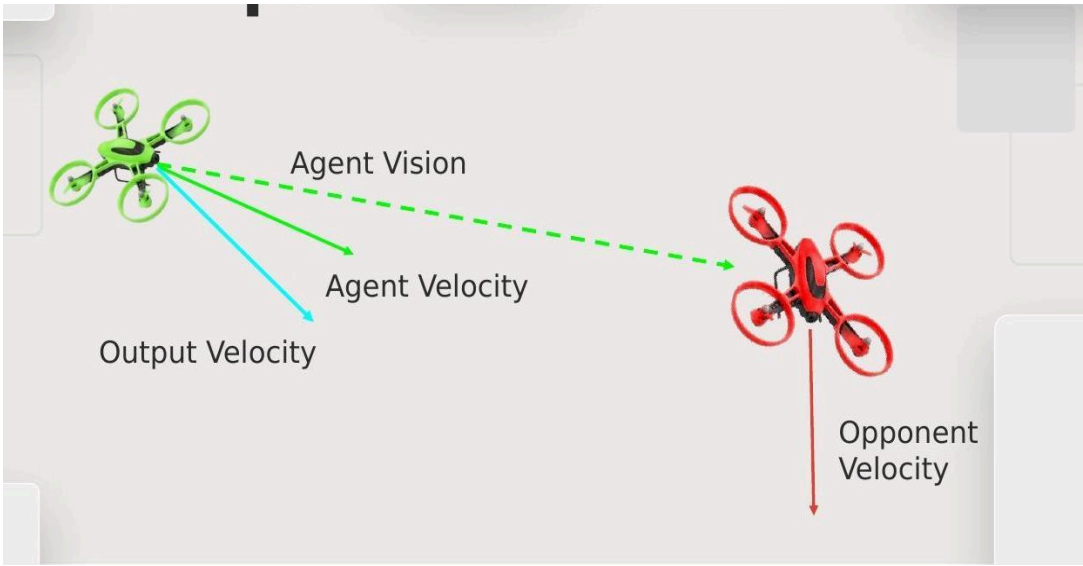

Figure 2. Observation geometry. The tracker (green) observes its own velocity, the opponent's velocity, and the opponent's relative position in its body frame. Its action is a 3D velocity command. The same nine-dimensional observation vector is fed at deployment, with AprilTag pose supplying the relative-position component and visual-flow differencing supplying the opponent velocity estimate.

Asymmetric, Vision-Limited Observation

Our initial training setup gave both agents full state observation. The result was uninformative. With complete knowledge of the evader's position the tracker became effectively omniscient, the evader could not generate a strategy that worked under any starting condition, and neither agent's policy improved past the first few hundred episodes. The fix was to restrict the tracker's observation to a cone-shaped field of view roughly approximating the geometry of a forward-facing camera on the deployment platform, with the relative-position component zeroed out whenever the evader fell outside the cone. This single modification reshaped the game. The tracker now had to expend effort keeping the evader inside its field of view instead of homing in on the opponent regardless of relative bearing, and the evader was suddenly given a concrete objective worth optimizing for, which was simply to break line of sight. The qualitative difference in behavior was the most striking single result of the project. An evader trained under full-observation collapsed onto a strategy of running directly away as a function of distance, while an evader trained against a tracker with a limited field of view developed feints, occlusion-seeking maneuvers, and direction reversals timed to the tracker's reaction lag.

Curriculum and Obstacle

Without a curriculum on the starting conditions of each episode, both policies converged onto trivial fixed strategies regardless of training duration. The evader learned to sprint along a vector directly away from the tracker, scaled by the current distance. The tracker learned to trail at a fixed offset and decelerate to match the evader's speed. Neither agent ever discovered a feint or a last-second dodge, because no episode in the training distribution required one. To force both agents off these solutions, we shrink the initial separation between the two drones progressively across training rounds, which pushes the engagement into a close-range regime in which a constant-velocity strategy is no longer sufficient for the evader and a fixed-offset trail is no longer sufficient for the tracker. The exact schedule, how aggressively the separation should shrink and at which round the obstacle should be introduced, was itself a tuning exercise; the values reported here are the ones that worked, after several earlier schedules either ramped difficulty too fast (collapsing the evader's policy) or too slowly (the tracker just got better at the easy regime without learning anything new). Partway through training, after the sixth round in our final configuration, we raise the evader's maximum speed by 10% and place a 2-meter-tall cylindrical obstacle at the arena center. The obstacle gives the evader a concrete geometric feature to exploit, since it can break the tracker's line of sight by transiting behind the cylinder, and it forces the tracker to learn occlusion-tolerant behavior rather than open-air pursuit.

Reward Shaping

The reward functions for both agents were tuned primarily to keep them off the degenerate solutions described above, rather than to reward any specific strategy directly. The tracker receives a positive reward proportional to negative-tanh of distance to the evader, which keeps the reward gradient nonzero at long range where a linear distance penalty would saturate. It also receives a positive shaping term proportional to the per-step closing rate, a bonus for keeping the evader near the center of its field of view, and a large terminal reward (+60 to +110, scaled inversely with episode length) on a successful capture inside 0.7 m, with a small standing penalty for excursions outside the 20 m arena boundary. The evader is rewarded for large distance from the tracker, for opening distance step-over-step, and for not holding a constant heading-and-speed across consecutive steps; the last term is what specifically suppresses the converge-to-straight-line failure mode. The smoothness term on both reward functions is structurally significant rather than cosmetic: it discourages high-frequency oscillation in the velocity command, which produces commanded trajectories the Tello's flight controller can execute without saturating, and is therefore a direct contributor to successful transfer rather than a refinement of the simulated visuals.

Training Procedure

Training proceeds in alternating self-play rounds. Within a round, one agent learns with PPO while the other is held frozen at its policy from the previous round and used only to generate opponent behavior. After a fixed number of timesteps in each phase, the roles flip. Because the learning agent always faces a competent but stationary opponent, the per-round reward curve oscillates by design: whichever agent is currently being optimized climbs against the frozen opponent before the advantage flips on the next swap. This oscillation is visible in Figure 7 and is the diagnostic we used to confirm the curriculum was producing adversarial pressure on both sides rather than allowing one agent to dominate the other indefinitely. The full run consists of 1,000 episodes of approximately 3,000 timesteps each, for three million timesteps in total. PPO updates run every 512 collected steps. Training took approximately five hours on a single NVIDIA RTX 3060.

Checkpoint management was reworked midway through development after an incident in which checkpoints were scattered across multiple folders and a combined-load step inadvertently loaded the wrong model, destroying roughly thirty hours of training progress. The current layout uses one save directory per run, containing the active tracker and evader networks, periodic timestamp-named checkpoints, the PPO log directory, and a small JSON file recording the round index and timestep count so that a resumed run begins from exactly the round and step at which it was interrupted.

The self-play loop is summarized in the pseudocode below.

```
initialize tracker_pi, evader_pi   # PPO, MLP [128,128], lr 5e-4, gamma 0.98
pretrain tracker_pi for K_warm steps
for round in 1 .. N_rounds:
    start_separation <- shrink_schedule(round)        # curriculum
    if round >= round_obstacle:
        enable cylinder obstacle
        evader.v_max <- 1.10 * evader.v_max_base
    freeze(evader_pi)                                 # train tracker
    for step in 1 .. K_per_round:
        o <- env.observe(tracker)                     # 9-dim relative obs
        a <- tracker_pi.act(o)                        # 3-D velocity
        env.step(a_tracker=a, a_evader=evader_pi.act())
        PPO.update_every(U_steps)
    freeze(tracker_pi); train evader_pi symmetrically
    checkpoint(tracker_pi, evader_pi, progress.json)
```

Metrics

We track three metrics during training and evaluation. The tag rate is the fraction of episodes the tracker wins, defined as G□ / N, where G□ is the number of episodes ending in a capture inside the catch radius and N is the total number of evaluation episodes. The

time-to-capture is the mean simulated time elapsed before the tracker closes inside the catch radius, averaged over caught episodes only. The evader survival time is the mean simulated time before capture, viewed from the evader's side; this was the metric we used during curriculum design to verify that the evader was actually becoming a harder target across rounds, rather than that the tracker was regressing against the previous-round opponent. Early in training, at roughly 4,000 of the first 30,000 timesteps, mean evader survival was approximately 8 seconds before capture. The monotonic increase of that quantity over subsequent rounds was the primary signal that the curriculum was operating as intended.

What Did Not Work

A handful of early approaches were attempted and dropped during the project, and each one shaped the final design enough to be worth documenting briefly. The first such dead end was the attempt to follow the AirSim reinforcement-learning tutorials directly. Sustained progress on AirSim turned out to be blocked by a combination of dependency problems: the airgym package would not install cleanly against current Python versions, the migration from gym to gymnasium broke substantial portions of the example code, and the Microsoft-maintained AirSim Python client had been deprecated. We spent roughly three weeks of the early calendar resolving these issues with almost no training to show for it, and ended up treating that block as the single biggest avoidable time sink of the year; it is what eventually motivated the migration of the training environment to PyBullet. A second early attempt gave the evader an explicit backward-flying behavior by negating the tracking velocity, on the hypothesis that an evader without a rear camera could still flee while observing the tracker. The behavior was unstable in PyBullet rollouts and inferior to the simpler yaw controller applied at deployment time, which we describe in Section V. A symmetric full-observation training run, discussed in the previous subsection, similarly produced an omniscient tracker and no real evasion behavior, and is what motivated the asymmetric observation design used in the final system.

Off-the-shelf GPS modules were investigated briefly as a source of absolute positioning indoors and rejected. Consumer-grade modules deliver positioning errors on the order of meters in open sky and considerably worse indoors, against a control system that needs centimeter-level precision over a 20 m arena. The other large block of calendar time, alongside the AirSim issues, was spent waiting on drone parts. Several of the components on the custom-build BOM had multi-week shipping windows, and the assembly itself took longer than we expected; looking back at it, the experience of physically building the airframe is what told us how much the deployment platform was going to constrain the simulation design, and we

would have benefited from doing the build earlier in the project, even just a pilot version, so the sim could have been designed around what the real drone could actually do.

V. Sim-to-Real Methodology

Intended Hardware: a Custom-Built Quadrotor

The original hardware plan was a custom-built 5-inch racing quadrotor with onboard compute. The full bill of materials, given in Appendix B, includes a carbon-fiber frame, a four-in-one 30 A electronic speed controller, brushless RS2205 motors, a Holybro Kakute flight controller, a 2.4 GHz ExpressLRS radio link, a 5S 1550 mAh lithium-polymer battery, an IMX219 8-megapixel camera, and an NVIDIA Jetson Orin Nano companion computer to run the policy and the perception pipeline on board. The intended power flow and signal flow are shown in Figure 3. The total cost of the build was approximately $1,240.

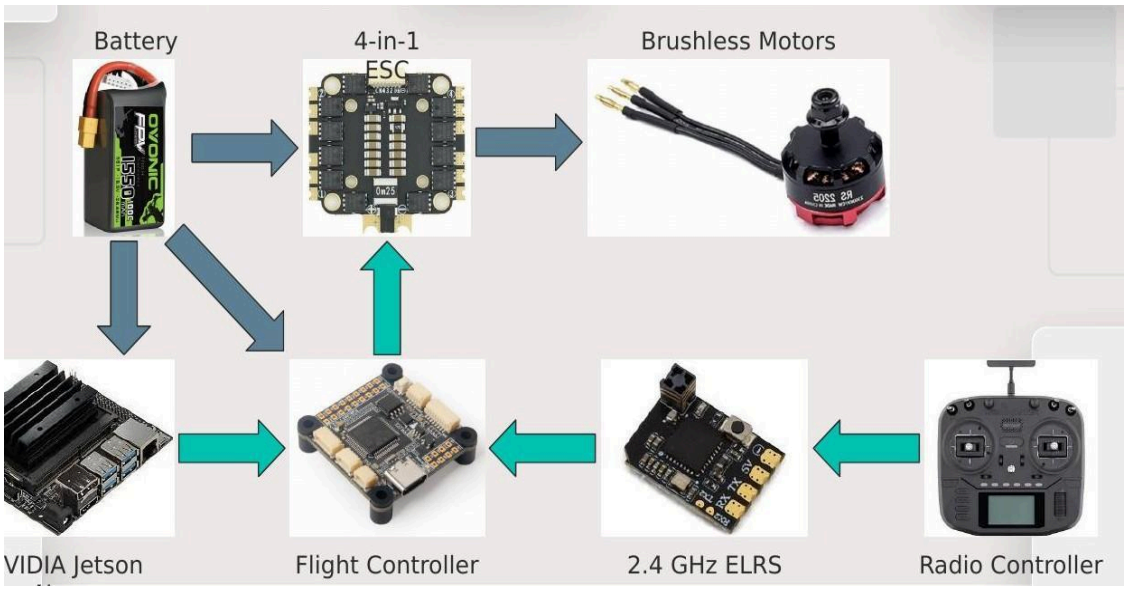

Figure 3. Custom quadrotor hardware diagram. The blue arrows indicate the power distribution path from the LiPo battery to the four-in-one ESC and onward to the brushless motors. The teal arrows indicate the signal path: the Jetson Nano computes velocity commands from camera input and forwards them to the flight controller, which receives independent manual commands from the ELRS radio receiver as a safety override.

During the final assembly of the airframe, a wiring fault during the power-distribution-board solder produced a short circuit that destroyed both the flight controller and the four-in-one ESC. Figure 4 shows the post-incident state of the two boards. The frame was salvageable, but the cost and lead time of replacing the destroyed electronics with the same components were beyond what remained in the project schedule. The decision was therefore to abandon the custom build and substitute a commercial platform for the demonstration phase.

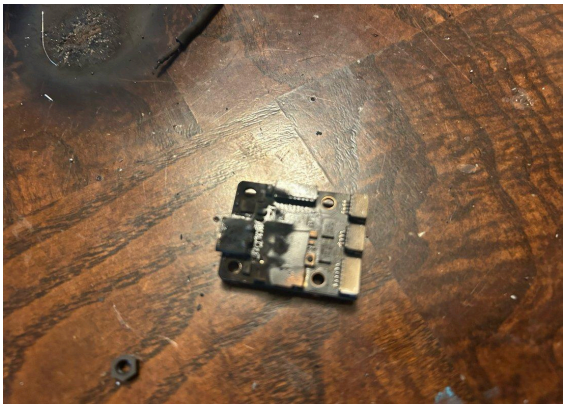

Figure 4. Burned flight controller and four-in-one ESC after the short circuit during final assembly of the custom airframe.

Pivot to the DJI Tello EDU

The replacement platform was the DJI Tello EDU, a small commercial quadrotor designed for educational and entry-level developer use (Figure 5). The Tello exposes a velocity-command interface and a 720p H.264 video stream through a documented Wi-Fi UDP protocol, both of which align with the input and output the trained policy expects. The trade-off is that the Tello has no onboard compute and only a forward-facing camera, which forces all perception and policy evaluation to run off-board on a host laptop and constrains the deployment to a forward-vision geometry. Its low price made it practical to perform dozens of real-world test flights during the deployment-side tuning, which we would not have attempted on a build worth ten times as much.

Figure 5. DJI Tello EDU, the replacement deployment platform. Mass 87 g, single forward-facing 720p camera, no onboard compute, Wi-Fi UDP control and video interface, unit cost approximately $130.

Testing Method

The evader has no source of absolute positioning and a full two-drone learned-vs-learned physical engagement was outside the time available, so physical testing exercised the tracker

policy alone. We mounted a printed AprilTag (family tag36h11, 10 cm side length) on a second DJI Tello EDU that served as the evader stand-in; this second drone was flown manually, providing a moving but human-controlled target for the autonomous tracker. The arrangement gives the tracker reliable relative pose whenever the tag lies inside the forward camera's field of view, and lets us defer marker-free aerial detection, which is outside the scope of this paper, to future work. The simulated arena boundary was reduced from the 60 m used during early training to 20 m, since indoor space, including the largest gymnasium available to us, does not provide 60 m of clear flight. Revalidating the trained policy in roughly the volume in which it would be deployed, rather than preserving the original training boundary, was a deliberate trade in favor of evaluation realism.

Closing the Transfer Gap

Optimizing a policy purely in simulation yields poor performance on physical hardware if the discrepancies between the simulator and the real system are not mitigated. The discrepancies here are caused primarily by three factors: variable latency in the Wi-Fi video stream and the AprilTag detection pipeline, abrupt commanded acceleration that the lightweight Tello cannot execute cleanly, and the absence of any rear-facing perception. We address each with a dedicated component placed between the policy output and the drone.

Of the three, latency is what we had to work hardest to compensate for. The H.264 video stream arrives at the host with non-trivial jitter and the detection pipeline runs at roughly 15 Hz against a 20 Hz control loop, so the policy observation is frequently stale by 50 to 100 ms by the time it reaches the network. We extrapolate the relative position of the evader forward in time between vision frames using the most recent smoothed velocity estimate, capped at a maximum prediction horizon of 500 ms, beyond which the predicted position is discarded and the last detected position is reused instead. The video reader and detector run on threads independent of the control loop, so detection latency cannot stall command transmission to the drone. During training we also injected a uniform jitter of ±30 ms into the simulator step time, on the theory that the policy ought to encounter variable observation delay before it encountered any real one. Action smoothness is enforced through a one-pole low-pass filter on the policy output with smoothing coefficient $\alpha = 0.15$, which we tuned by hand across a few flights: lower values caused visible jitter in the Tello's commanded velocity, and higher values introduced enough lag that the tracker fell behind on sharp direction changes. Field-of-view loss is handled by an auxiliary yaw controller that rotates the Tello in body-frame yaw to keep the AprilTag near the image center; the controller is a simple proportional law with gain 90 deg/s/rad of horizontal bearing error and a 0.04 rad deadband. The yaw command rides in

parallel with the velocity command from the network, so rotation does not require the network itself to plan for keeping the target in view.

Figure 6 shows the perception output observed during a typical real-world flight. The detection of a phone-held marker stand-in (used in early testing prior to the AprilTag-mounted Tello arrangement) is shown for clarity at confidence 0.67, with the screen-space velocity vector drawn in magenta from the detection center and the tracker's commanded direction crosshaired at the box center. The same pipeline operates on the AprilTag-mounted Tello during the formal tracking trials reported below.

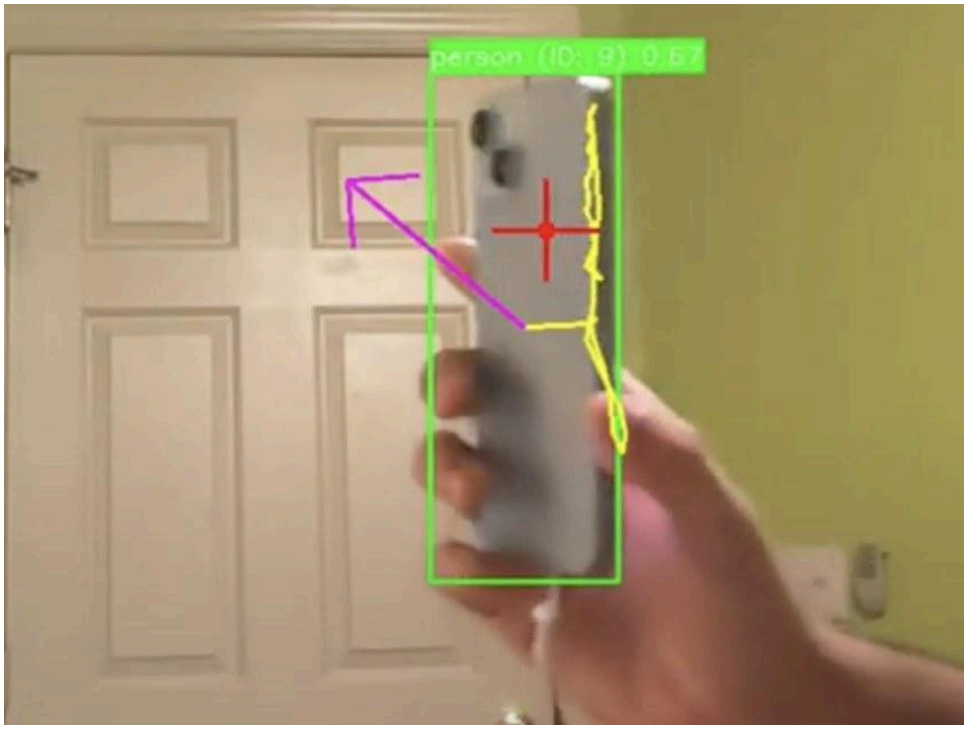

Figure 6. Onboard perception output from the Tello camera. The detection (green box) reports confidence 0.67. The magenta arrow is the smoothed evader velocity vector estimated from successive detections, and the red crosshair indicates the tracker's commanded direction of motion. The same 9-dimensional observation vector is then assembled and passed into the trained network, identical in form to the one used during PyBullet training.

VI. Results

By the end of training, the simulation had settled into the adversarial equilibrium we had been aiming for, with the tracker reaching a final average tag rate of 95% against an evader that was adapting in parallel; Figure 7 plots per-episode reward and tag rate over the full three-million-timestep run. The reward curves oscillate between training phases in roughly the pattern that self-play predicts, with the active agent climbing against the frozen opponent before the curve inverts on the next swap. That oscillation becomes more pronounced after the

obstacle is introduced midway through training, with both the magnitude and the frequency of the swings visibly increasing, and the final tag-rate curve settles near 0.95 with small late-stage perturbations that we interpret as the system responding to fresh opponent variations during the closing rounds rather than as catastrophic forgetting.

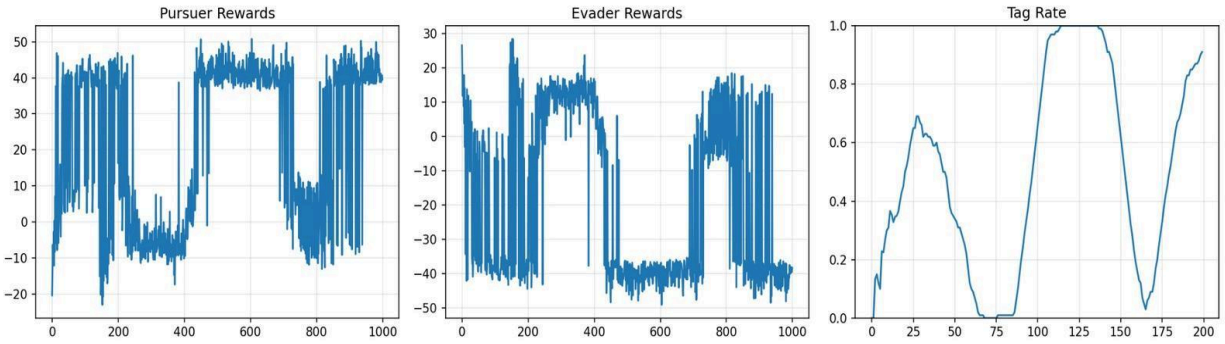

Figure 7. Self-play training curves. Left: per-episode pursuer reward over 1,000 episodes, showing oscillation between training phases. Center: evader reward over the same window, mirror-symmetric in expectation. Right: average tag rate over evaluation episodes per round. The dip mid-training corresponds to the introduction of the obstacle and the evader-speed increase; the tracker recovers to a final tag rate of approximately 0.95.

Qualitatively, the post-curriculum evader exhibits behaviors that no part of the reward function specifies. After the obstacle is introduced, the evader learns to break the tracker's line of sight by transiting behind the cylinder, and once line of sight is broken it reverses heading sharply rather than continuing along its previous trajectory. It also learns short-amplitude lateral feints under close pursuit, alternating direction at roughly the same time scale as the tracker's reaction lag. Neither of those strategies is hand-coded anywhere in the reward function or in the environment, and we read their appearance as a sign that the adversarial pressure from a concurrently improving opponent really is what drives the policies to discover non-trivial tactics, rather than the reward shaping doing the work on its own.

How well the system actually transferred was the part of the project we were most uncertain about going in, and it is also the part where the simulation-side results matter least on their own. The trained tracker, with no further training or fine-tuning of any kind, was loaded onto the Tello deployment script and tested across several dozen indoor flights against a second DJI Tello carrying the printed AprilTag and flown manually as a moving target. The controller tracked this target Tello through direction reversals and through brief occlusions; recovery from a lost-tag state took one to two seconds on average. The dominant failure mode in real flight, distinct from the trivial-strategy collapse seen in early simulation, was loss of detection at extreme angles where the tag exits the camera's field of view faster than the yaw controller can rotate to compensate. In those cases the search behavior described in Section V rotates the drone slowly toward the tag's last-known bearing until detection re-acquires, which

it usually does within a second or two so long as the tag has not actually left the room. The simulation-versus-hardware contrast, along with what fails in each regime, is laid out in Table 1.

Table 1. Simulation versus real-world deployment characteristics.

Aspect	Simulation	Real DJI Tello
Opponent state	Exact relative position and velocity at every step	AprilTag pose at ~15 Hz, velocity estimated by differencing
Observation latency	0 ms (single step)	50–100 ms typical, capped at 500 ms predicted
Tracking outcome	95% tag rate after 3M timesteps	Stable tracking of a moving target across dozens of trials
Dominant failure mode	Trivial-strategy collapse (resolved by curriculum)	Loss of detection at extreme yaw rates
Recovery	N/A	Search behavior rotates toward last-seen bearing
Output	Continuous 3D velocity command	Filtered velocity + parallel yaw command

VII. Limitations

The work has several limitations that affect the strength of the claims that can be drawn from it. The physical target is a printed AprilTag mounted on a second drone rather than a second autonomous learned policy, so what we demonstrate experimentally is learned visual tracking of a fiducial-marked target, not a complete two-agent engagement between two learned policies on hardware. Everything downstream is gated on the tag being detected; under low light, at long range, or when the tag exits the forward field of view too quickly for the yaw controller to compensate, detection fails and the inter-frame velocity predictor only covers a brief gap before the drone enters its search behavior. The Tello has limited thrust and only forward-facing perception, which caps the aggressiveness of the maneuvers the policy is willing to attempt without losing the target. Indoor space available to us required reducing the simulated arena from 60 m to 20 m, so the policy is validated against a smaller operating envelope than the largest one it trained against. Our experimental evaluation rests on the simulated tag rate and on qualitative behavior across a few dozen real flights rather than on a large statistically controlled sweep; each physical flight consumes battery time, and a properly sized study would require more time and equipment than were available. The destroyed

custom airframe and the AirSim dependency problems also ate calendar time that would otherwise have gone into additional ablations and a quantitative real-world evaluation.

VIII. Conclusion

We built and trained an adversarial pursuit-evasion system on inexpensive hardware, end to end, with both the tracker and the evader acquiring their behavior through PPO self-play inside a custom PyBullet environment rather than from any hand-written control rules. The combination of an asymmetric, vision-limited observation, a curriculum that ramps evader speed and introduces an obstacle midway through training, and a reward function with an explicit smoothness term turned out to be what kept the policies from collapsing onto the trivial fixed-strategy solutions they otherwise converged to. Trained against an adaptive evader over three million timesteps, the tracker reaches a 95% tag rate in simulation, and the same network, with no retraining of any kind, runs on a $130 DJI Tello and tracks an AprilTag-mounted target drone in indoor flight at 20 Hz, with the velocity predictor, the low-pass action filter, and the parallel yaw controller compensating for the latency and field-of-view differences between the simulator and a real Wi-Fi-bound quadrotor. If there is a single thing we walked away from the project knowing, it is that the sim-to-real gap for adversarial closed-loop control on cheap hardware is much less about the learning algorithm than about the perception loop and the observation interface, specifically about keeping observations relative to the platform and keeping the per-step observation latency small enough that the policy is acting on something close to current information about the opponent.

IX. Future Work and Recommendations

- Replace the AprilTag with a learned onboard detector. A YOLOv5-class network running on a Jetson or Raspberry Pi companion computer would let the system track an actual drone instead of a printed marker, and would remove the cooperative-fiducial assumption that is currently the largest gap between this system and a fielded counter-UAV one.
- Fly two physical drones against each other. Doing so would exercise the deployed evader policy, which is currently unexercised on hardware, and would constitute the first physical adversarial engagement between two learned policies in this project line.
- Apply systematic domain randomization across observation noise, randomized vehicle dynamics, and wider latency-jitter ranges. The current policy handles the latency we

measured on the Tello well, but a domain-randomized policy would generalize across hardware platforms without requiring platform-specific tuning.

- Add accurate indoor positioning, e.g. via a motion-capture system or top-down ceiling cameras. Consumer-grade GPS modules do not deliver the precision required at indoor scales; an alternative positioning source would enable closed-loop quantitative evaluation of the evader policy on hardware.
- Conduct a quantitative real-world evaluation across many randomized trials, reporting tag rate, time-to-capture, and survival time with confidence intervals, rather than the qualitative flight behavior we are currently limited to.
- Extend to multi-pursuer or team-versus-team scenarios. The multi-agent reinforcement learning literature suggests that pursuit-evasion at scale produces qualitatively richer emergent behaviors than the two-agent case, and the existing system would extend naturally

X. Materials

Item	Qty	Notes
DJI Tello EDU	1+	Physical platform; controlled from Python via djitellopy
Printed AprilTag (tag36h11, 10 cm)	1	Relative-pose target standing in for the evader drone
Host laptop with NVIDIA RTX 3060	1	PPO self-play training (PyTorch, Stable-Baselines3, PyBullet)
Custom FPV quadrotor parts	set	Frame, 4-in-1 ESC, brushless motors, FC, ELRS radio, Jetson Orin Nano; see Appendix B
Software stack	—	Python 3.11, PyBullet, Stable-Baselines3, Gymnasium, OpenCV, pupil-apriltags, djitellopy, PyAV

References

[1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017. Available: https://arxiv.org/abs/1707.06347.

[3] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable Reinforcement Learning Implementations," J. Machine Learning Research, vol. 22, no. 268, pp. 1–8, 2021. Available: https://stable-baselines3.readthedocs.io.

[4] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "AirSim: High-Fidelity Visual and Physical Simulation for Autonomous Vehicles," in Field and Service Robotics, 2017. Available: https://github.com/microsoft/AirSim.

[5] E. Coumans and Y. Bai, "PyBullet, a Python module for physics simulation for games, robotics and machine learning," 2016–2021. Available: https://pybullet.org.

[6] D. Fuentes-Escó, "djitellopy: DJI Tello drone Python interface." Available: https://github.com/damiafuentes/DJITelloPy.

[7] E. Olson, "AprilTag: A robust and flexible visual fiducial system," in Proc. IEEE ICRA, 2011. Available: https://april.eecs.umich.edu/software/apriltag.

[8] B. Baker, I. Kanitscheider, T. Markov, Y. Wu, G. Powell, B. McGrew, and I. Mordatch, "Emergent Tool Use from Multi-Agent Autocurricula," arXiv:1909.07528, 2019. Available: https://arxiv.org/abs/1909.07528.

[9] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments" (MADDPG), NeurIPS, 2017. Available: https://github.com/openai/maddpg.

[10] D. Wang, T. Fan, T. Han, and J. Pan, "A Two-Stage Reinforcement Learning Approach for Multi-UAV Collision Avoidance Under Imperfect Sensing," arXiv. Available: https://arxiv.org/html/2408.10215v1.

[11] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," Nature, vol. 620, pp. 982–987, Aug. 2023. Available: https://doi.org/10.1038/s41586-023-06419-4.

[12] G. Jocher et al., "YOLOv5: A state-of-the-art real-time object detection system," Ultralytics, 2020–2023. Available: https://github.com/ultralytics/yolov5.

Project Deliverables

GitHub repo: rn-om-syslab/syslab

Appendix

A. Source Code

Full project source code, including the PyBullet pursuit-evasion environment with the custom Gymnasium tracker and evader subclasses, the PPO self-play training loop with checkpoint and resume logic, the Tello deployment script with AprilTag detection and inter-frame velocity prediction, and the trained model checkpoints and training logs, is available in the project GitHub repository linked above. The repository also includes the demonstration scripts and the configuration files used to produce the figures in this paper.

B. Bill of Materials (Custom Airframe, As Originally Planned)

Part	Qty	Unit	Total
Frame (5")	2	$22.99	$45.98
4-in-1 Electronic Speed Controllers	2	$39.99	$79.98
Brushless Motors (4-pack)	2	$35.99	$71.98
Propellers (16-pack)	1	$17.59	$17.59
Holybro Kakute Flight Controller	2	$38.99	$77.98
Power Distribution Board	2	$11.96	$23.92
Jetson Nano Carrier Board	2	$69.99	$139.98
Jetson Orin Nano (companion computer)	2	$249.99	$499.98
IMX219-83 8MP 3D Camera	2	$36.00	$72.00
5S LiPo Battery (18.5 V nominal, 1550 mAh)	2	$30.00	$60.00
ExpressLRS XR2 Nano Receiver	2	$49.99	$99.98
Telemetry Module	2	$25.00	$50.00
Total Estimated Cost			**$1,239.37**

After the short circuit during assembly destroyed the flight controller and ESC, the demonstration platform was substituted with a DJI Tello EDU.